Job Posting Autocomplete and Recruiter Postings

- **Purpose of AI Use**: Improving the Job Posting with Autocomplete & Detailed Dropdown
- **AI-Suggested Content**: Matching autocomplete feature as done in the Experience seciton for job seekers.
- **Validation**: Tested and worked, with some quirks due to the API being slow
- **Decision**: Accepted and Iterated
- **Reflection**: AI Helped copy an already done feature to make it universal for the webapp.
- **Responsible Person**: Paul Fourment

Prompt 1

Checking my teammate's branch: for the job posting, can you apply the same APIs we used for profile management to the create job posting form, with company name, location, etc.?

Response 1

The job posting form now uses the same company and city autocomplete as the profile.

- **Company field**: suggestions with logo and website as you type. Picking one saves the company's website domain with the

posting.
- **Location field**: city suggestions, plus "Remote".
- **Logos**: the job list and the posting page show the company's logo next to the title, or its first letter if no suggestion was picked. Existing postings show the letter until edited.
- **Validation**: a malformed domain is dropped and never saved, with a unit test added next to the teammate's existing tests.
- Employment type and work mode already used the shared lists from the profile, so they were left as they were.

SQL to run in the Supabase SQL Editor

```
alter table public.job_posting add column company_domain
```

Prompt 2

When I hit create, the company is autofilled with "Acme Inc"; it should be empty. And for location, if I enter a city, "Remote" is the only option and is always shown while typing.

Response 2

- **Company**: a new posting now starts empty (the prefill from the recruiter's profile was removed).
- **Location**:
  - "Remote" only appears when the typed text is the start of it ("r", "rem"), not whenever it contains the same letters.
  - A "Searching…" row shows while the city search is running, instead of an empty or misleading list.
  - Typing "Montr" shows Montreal, Montreal West and Montréal-Est once the search returns.
- The fix is in the shared autocomplete code, so the profile and

onboarding location fields get the same behaviour.

Testing: checked in the browser on a temporary preview page (since removed). The city search itself still takes about 2.5 seconds on the free Photon service; "Searching…" makes the wait visible.

Prompt 3

Can we separate YOUR job postings from the general ones?

Response 3

Recruiters get two tabs on the Jobs page: **All postings** and **Your postings**.

- **Your postings** (`/jobs?mine`) lists only the jobs that recruiter created, newest first, with the heading "Your job postings" and a "Create your first posting" link when empty.
- **Job seekers** see no tabs, only the general list; adding `?mine` to the URL does nothing for them.
- The filter runs on the server. The recruiter card on the home page links to both lists, and after deleting a posting the recruiter lands back on their own list.

Prompt 4

There should be an Edit button next to it instead of having to click on the posting to view and edit it.

Response 4

Each posting the recruiter owns now has an **Edit** button on its card, in both tabs, going straight to the editor.

- Only shown on the logged-in recruiter's own postings; other recruiters' postings and job seekers' lists don't show it.
- The rest of the card still opens the posting.
- Screen readers hear which posting each button edits (e.g. "Edit Software Developer Intern").
- Also fixed an existing layout bug: "Create a posting" and "Edit posting" were stretching across the full width because of conflicting width classes. They now sit beside the heading on wider screens and stay full-width on phones.

Testing: checked in the browser with sample postings on a temporary preview page (since removed).

Resume Management (US-10, US-11) and AI Resume Features

- **Purpose of AI Use**: Implement Resume Upload and Management, and Starting AI Generative Feature with Resume
- **AI-Suggested Content**: The resume upload and management feature, and a way to run the AI generative feature with either google cloud or local model.
- **Validation**: Tested resume upload, & Tested AI feature with local model.
- **Decision**: Accepted & Still Working on AI feature
- **Reflection**: AI helped decide on not 1 but 2 possible AI generative features, and a way to use it locally without havign to pay for a

cloud model.
- **Responsible Person**: Paul Fourment

Prompt 1

I have opened a new branch for the last profile feature we need for job seekers. Here are the user stories:

(*US-10 pasted: upload PDF resumes, stored securely, confirmation after upload, invalid formats rejected. Tasks 10.1–10.4: resume database/ storage model, upload API, upload UI, file type and size validation.*)

(*US-11 pasted: view, replace and delete uploaded resumes, changes reflected immediately. Tasks 11.1–11.4: resume list page, replacement, deletion, confirmation dialog for deletion.*)

With this feature, I'd like to look into having a tool autofill the profile when uploading a resume, to help fill in things like experience and education, with the dates and bullet points as well.

Another feature to look into: since the project requires a Generative AI feature, I was thinking about a Resume Reviewer. However, we want to avoid paying for an API, or we could run it locally. I know about https://github.com/interviewstreet/hiring-agent

(*Project requirement pasted: at least one Generative AI feature, plus one additional original feature; students may use an external AI API or run a model locally.*)

Response 1

Resume upload and management (US-10, US-11)

- **`/resumes` page** for job seekers, with a "Resumes" link in the sidebar:
  - **Upload** a PDF with a confirmation message.
  - **List** each resume with name, size and date; **View** opens the PDF in a new tab.
  - **Replace** swaps the file in place; **Delete** asks for confirmation in a glass dialog (focus starts on Cancel, Esc closes it). The list updates immediately after each action.
- **Limits**: PDF only, up to 4 MB each (hosts like Vercel reject uploads over about 4.5 MB), 5 resumes per person.
- **File checks (Task 10.4)**: the server checks the file type, the `.pdf` extension, and that the content actually starts like a PDF, so a Word file renamed to `.pdf` is rejected. Covered by unit tests.
- **Security ("stored securely")**: files go to a private bucket and can't be reached by URL, only through our server; only the owner can open a resume (everyone else gets "not found"); the storage path never contains anything the user typed; deleting the account also deletes the resume files.
- **Profile**: the resume placeholder became the resume list with a Manage link, and uploading a resume counts toward profile completeness.
- Recruiters can't see resumes yet; that belongs with job applications.

SQL to run in the Supabase SQL Editor

```
create table public.resume (
  id           uuid primary key default gen_random_uuid(),
  user_id      text not null references public."user"(id) on delete cascade,
  file_name    text not null,
  size_bytes   integer not null,
  storage_path text not null unique,
  created_at   timestamp not null default now(),
  updated_at   timestamp not null default now()
);
create index resume_user_id_idx on public.resume(user_id);
alter table public.resume enable row level security;
```

Storage → New bucket: `resumes`, with **Public bucket off** (optionally 4

MB limit, `application/pdf` only).

The two AI ideas

- **hiring-agent**: a Python program (MIT licensed) that turns a resume PDF into text, asks a model for structured data, then scores it. It's built for recruiters ranking many applicants, and its own README links to articles showing the same resume scoring anywhere from 74 to 90 across runs. Suggestion: borrow its approach (PDF → text → model → structured data) inside our own server and credit it, instead of running a second Python service.
- **Running it free**: Ollama with a local Qwen model can return answers in a fixed data format, which both features need; the assignment allows a local model and resumes never leave the machine. The catch: it only works on a machine running Ollama, so the code should make switching to a free hosted tier (Google Gemini or Groq) a settings change, and hide the AI buttons when no model is configured.
- **Autofill**: the model returns headline, about, skills, and each job and school with dates and bullet points; the user ticks what to import, and every item goes through the same validation as manual entries.
- **Reviewer**: written feedback (strengths, improvements per section, rewritten example bullets, missing keywords), optionally against a job posting. Written feedback rather than a score, given the score variance above. This covers the required "AI-assisted resume feedback" feature; autofill could be the additional original feature.

Prompt 2

Do you think we can use a free hosted tier and local Ollama with that quick switch idea? And provide documentation for teammates. Let's do both?

Response 2

Yes: Ollama, Google Gemini and Groq all support the same "OpenAI-compatible" API, so one piece of code talks to any of them, chosen by three lines in `.env`:

```
AI_BASE_URL="http://localhost:11434/v1"
AI_MODEL="qwen3.5:9b-mlx"
AI_API_KEY=""
```

When they're empty, the AI buttons don't appear and the rest of the app works normally.

AI client

- Asks the model for a fixed JSON structure, with a fallback for providers that don't support the stricter format, and turns off the model's "thinking" mode (much faster on the local model).
- Clear error messages per case (model not reachable, e.g. "check that Ollama is running"; request rejected; unusable answer).
- PDF text is extracted with the `unpdf` package; scanned PDFs with no readable text get a clear error.
- The prompts tell the model to treat the resume as data (ignore instructions inside it) and never invent information. Unit tests use a faked network, so they run without a model.

Resume Reviewer (Review button on each resume)

- Overall impression, what works, what to improve section by section, up to three rewritten bullet points, and keywords worth adding.
- Can be tailored to any job posting, which adds a "fit for this job" summary and what the posting asks for that the resume lacks.
- The review is saved, so reopening the page doesn't rerun the model. Includes an "AI can be wrong" note.

Fill profile from resume (Fill profile button on each resume)

- The model reads the resume and shows a checklist of what it found; nothing changes until "Add to profile".
- Fields that would overwrite existing data are unticked and marked "Replaces: …"; jobs and schools already on the profile are greyed out; only new skills are offered.
- Every item passes the same validation as the manual profile forms, so a wrong date from the model is dropped, not saved.

Testing on the local model (M3 Pro)

- Extraction on a sample resume was fully correct: both jobs with dates and bullet points, both schools, 13 skills.
- First review run: the tailored rewrites added things the resume never said (e.g. "Docker"). The prompt was tightened so rewrites must describe exactly the same work, and the rerun stayed faithful.
- The model said 2026 dates were "in the future", so today's date was added to the prompt.
- About 90–120 seconds per request locally; the pages say "this can take a minute or two". A hosted model should take a few seconds.
- Gemini and Groq weren't live-tested (no keys).

Teammate guide (`docs/ai-setup.md`, linked from the README): setup for all three options with a comparison table (cost, privacy, speed), rules for handling API keys, a troubleshooting table matching each error message to its fix, a short "how it works" section, and credit to hiring-agent for the approach.

SQL to run in the Supabase SQL Editor

```
alter table public.resume
  add column parsed      jsonb,
  add column parsed_at   timestamp,
  add column review      jsonb,
  add column reviewed_at timestamp;
```

Resume Management (US-10, US-11) and AI Resume Features

- **Purpose of AI Use**: Implement Resume Upload and Management, and Starting AI Generative Feature with Resume
- **AI-Suggested Content**: The resume upload and management feature, and a way to run the AI generative feature with either google cloud or local model.
- **Validation**: Tested resume upload, & Tested AI feature with local model.
- **Decision**: Accepted & Still Working on AI feature
- **Reflection**: AI helped decide on not 1 but 2 possible AI generative features, and a way to use it locally without havign to pay for a cloud model.
- **Responsible Person**: Paul Fourment

Prompt 1

I have opened a new branch for the last profile feature we need for job seekers. Here are the user stories:

(*US-10 pasted: upload PDF resumes, stored securely, confirmation after upload, invalid formats rejected. Tasks 10.1–10.4: resume database/ storage model, upload API, upload UI, file type and size validation.*)

(*US-11 pasted: view, replace and delete uploaded resumes, changes reflected immediately. Tasks 11.1–11.4: resume list page, replacement, deletion, confirmation dialog for deletion.*)

With this feature, I'd like to look into having a tool autofill the profile when uploading a resume, to help fill in things like experience and education, with the dates and bullet points as well.

Another feature to look into: since the project requires a Generative AI

feature, I was thinking about a Resume Reviewer. However, we want to avoid paying for an API, or we could run it locally. I know about https://github.com/interviewstreet/hiring-agent

(*Project requirement pasted: at least one Generative AI feature, plus one additional original feature; students may use an external AI API or run a model locally.*)

Response 1

Resume upload and management (US-10, US-11)

- **`/resumes` page** for job seekers, with a "Resumes" link in the sidebar:
  - **Upload** a PDF with a confirmation message.
  - **List** each resume with name, size and date; **View** opens the PDF in a new tab.
  - **Replace** swaps the file in place; **Delete** asks for confirmation in a glass dialog (focus starts on Cancel, Esc closes it). The list updates immediately after each action.
- **Limits**: PDF only, up to 4 MB each (hosts like Vercel reject uploads over about 4.5 MB), 5 resumes per person.
- **File checks (Task 10.4)**: the server checks the file type, the `.pdf` extension, and that the content actually starts like a PDF, so a Word file renamed to `.pdf` is rejected. Covered by unit tests.
- **Security ("stored securely")**: files go to a private bucket and can't be reached by URL, only through our server; only the owner can open a resume (everyone else gets "not found"); the storage path never contains anything the user typed; deleting the account also deletes the resume files.
- **Profile**: the resume placeholder became the resume list with a Manage link, and uploading a resume counts toward profile completeness.
- Recruiters can't see resumes yet; that belongs with job applications.

SQL to run in the Supabase SQL Editor

```
create table public.resume (
  id           uuid primary key default gen_random_uuid(),
  user_id      text not null references public."user"(id),
  file_name    text not null,
  size_bytes   integer not null,
  storage_path text not null unique,
  created_at   timestamp not null default now(),
  updated_at   timestamp not null default now()
);
create index resume_user_id_idx on public.resume(user_id);
alter table public.resume enable row level security;
```

Storage → New bucket: `resumes`, with **Public bucket off** (optionally 4 MB limit, `application/pdf` only).

The two AI ideas

- **hiring-agent**: a Python program (MIT licensed) that turns a resume PDF into text, asks a model for structured data, then scores it. It's built for recruiters ranking many applicants, and its own README links to articles showing the same resume scoring anywhere from 74 to 90 across runs. Suggestion: borrow its approach (PDF → text → model → structured data) inside our own server and credit it, instead of running a second Python service.
- **Running it free**: Ollama with a local Qwen model can return answers in a fixed data format, which both features need; the assignment allows a local model and resumes never leave the machine. The catch: it only works on a machine running Ollama, so the code should make switching to a free hosted tier (Google Gemini or Groq) a settings change, and hide the AI buttons when no model is configured.
- **Autofill**: the model returns headline, about, skills, and each job and school with dates and bullet points; the user ticks what to import, and every item goes through the same validation as manual entries.
- **Reviewer**: written feedback (strengths, improvements per section,

rewritten example bullets, missing keywords), optionally against a job posting. Written feedback rather than a score, given the score variance above. This covers the required "AI-assisted resume feedback" feature; autofill could be the additional original feature.

Prompt 2

Do you think we can use a free hosted tier and local Ollama with that quick switch idea? And provide documentation for teammates. Let's do both?

Response 2

Yes: Ollama, Google Gemini and Groq all support the same "OpenAI-compatible" API, so one piece of code talks to any of them, chosen by three lines in `.env`:

```
AI_BASE_URL="http://localhost:11434/v1"
AI_MODEL="qwen3.5:9b-mlx"
AI_API_KEY=""
```

When they're empty, the AI buttons don't appear and the rest of the app works normally.

AI client

- Asks the model for a fixed JSON structure, with a fallback for providers that don't support the stricter format, and turns off the model's "thinking" mode (much faster on the local model).
- Clear error messages per case (model not reachable, e.g. "check that Ollama is running"; request rejected; unusable answer).
- PDF text is extracted with the `unpdf` package; scanned PDFs

with no readable text get a clear error.
- The prompts tell the model to treat the resume as data (ignore instructions inside it) and never invent information. Unit tests use a faked network, so they run without a model.

Resume Reviewer (Review button on each resume)

- Overall impression, what works, what to improve section by section, up to three rewritten bullet points, and keywords worth adding.
- Can be tailored to any job posting, which adds a "fit for this job" summary and what the posting asks for that the resume lacks.
- The review is saved, so reopening the page doesn't rerun the model. Includes an "AI can be wrong" note.

Fill profile from resume (Fill profile button on each resume)

- The model reads the resume and shows a checklist of what it found; nothing changes until "Add to profile".
- Fields that would overwrite existing data are unticked and marked "Replaces: …"; jobs and schools already on the profile are greyed out; only new skills are offered.
- Every item passes the same validation as the manual profile forms, so a wrong date from the model is dropped, not saved.

Testing on the local model (M3 Pro)

- Extraction on a sample resume was fully correct: both jobs with dates and bullet points, both schools, 13 skills.
- First review run: the tailored rewrites added things the resume never said (e.g. "Docker"). The prompt was tightened so rewrites must describe exactly the same work, and the rerun stayed faithful.
- The model said 2026 dates were "in the future", so today's date was added to the prompt.
- About 90–120 seconds per request locally; the pages say "this can take a minute or two". A hosted model should take a few seconds.
- Gemini and Groq weren't live-tested (no keys).

Teammate guide (`docs/ai-setup.md`, linked from the README):

setup for all three options with a comparison table (cost, privacy, speed), rules for handling API keys, a troubleshooting table matching each error message to its fix, a short "how it works" section, and credit to hiring-agent for the approach.

SQL to run in the Supabase SQL Editor

```
alter table public.resume
  add column parsed      jsonb,
  add column parsed_at   timestamp,
  add column review      jsonb,
  add column reviewed_at timestamp;
```